MOSCOW AVIATION INSTITUTE
(NATIONAL RESEARCH UNIVERSITY)

Institute No. 8 “Computer Science and Applied Mathematics”
Department 806 “Computational Mathematics and Programming”

Laboratory work No. 2
Course “Computer Graphics”

Camera and Lighting

Student: M. A. Khorokhorin
Group: M8O-306BV-24
Teacher: V. D. Bakharev

Moscow, 2026

Task

Goal: implement a movable camera and lighting with the Blinn-Phong model. The program runs in real time on Vulkan, GLFW and ImGui.

Variant 4: the truncated regular tetrahedron from the first lab. The base task without extra tasks is done: the object is drawn relative to the camera with one directional light; the camera, the light, the Blinn-Phong material and the ambient light are changed in the ImGui interface.

Method

Normals. The object is built as in the first lab, but a vertex has a normal attribute instead of a color. Normals are generated procedurally: the normal of a face is the normalized cross product $(b - a) \times (c - a)$ of two of its edges and points outwards. Faces do not share vertices, so every face has its own normal and the lighting is flat.

Camera. The camera position is a point p, its rotation is given by the yaw and pitch angles, the view direction is f = (cos pitch · sin yaw, sin pitch, −cos pitch · cos yaw). The view matrix V = lookAt(p, p + f, up) is the inverse of the matrix that places the camera into the world. The perspective matrix is built with glm::perspectiveRH_ZO with the field of view from the interface and the Y axis flipped for Vulkan.

Lighting is computed in the fragment shader in world space. The normal is moved to world space with the normal matrix $(M^{-1})^T$. For the directional light l = −d, where d is the direction of the rays, v is the direction to the camera, h = (l + v) / |l + v| is the half vector. The final color is C = A · k_a · albedo + (albedo · max(n · l, 0) + specular · max(n · h, 0)^shininess) · L, where A and k_a are the ambient color and intensity, L is the light color multiplied by its intensity, albedo, specular and shininess are the material parameters. The specular term is skipped when the surface faces away from the light ($n \cdot l \leq 0$).

Vulkan objects. The vertex and index buffers and one uniform buffer are allocated with Vulkan Memory Allocator. The uniform buffer holds the model matrix, the view-projection matrix, the normal matrix, the camera position, the light direction, color and intensity, the ambient light and the material; every field is a mat4 or a vec4, so the std140 layout matches the C++ structure without extra padding. The buffer is described by a VkDescriptorSetLayout with the vertex and fragment stages, the descriptor set is allocated from a VkDescriptorPool. The pipeline is the same as in the first lab, but the second vertex attribute is the normal.

Results

The figures show the object with different parameters: the interface window with the camera, light, ambient light and material is on the left.

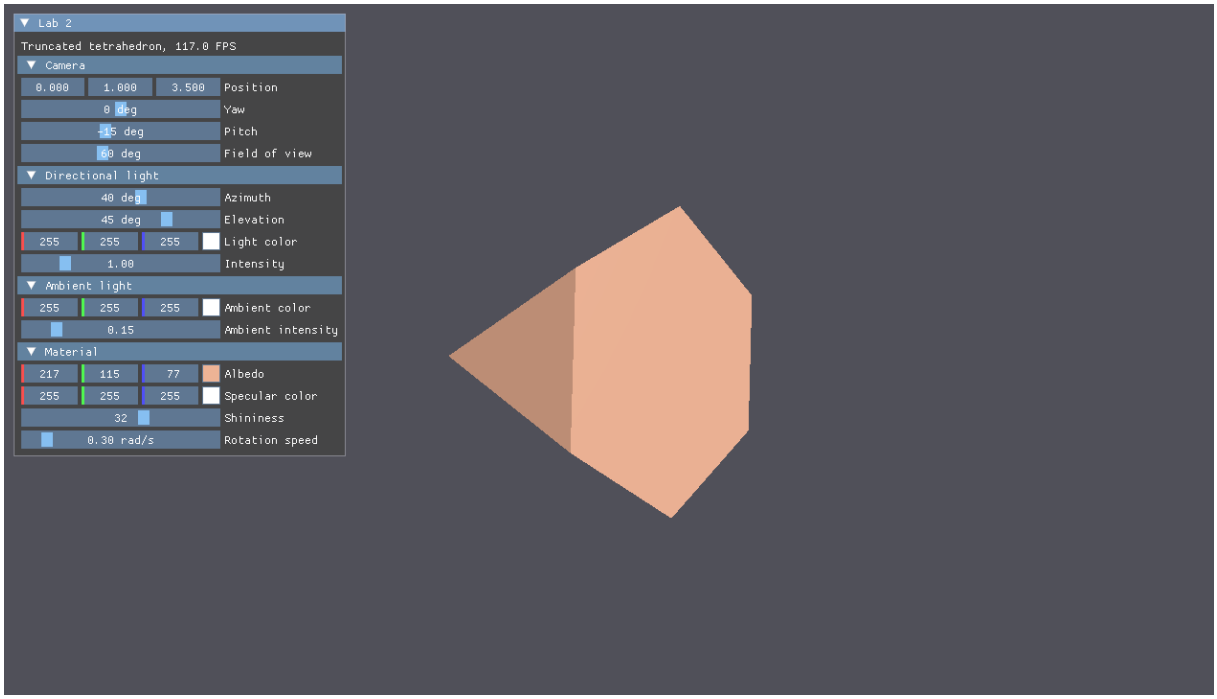


Figure 1 — default parameters

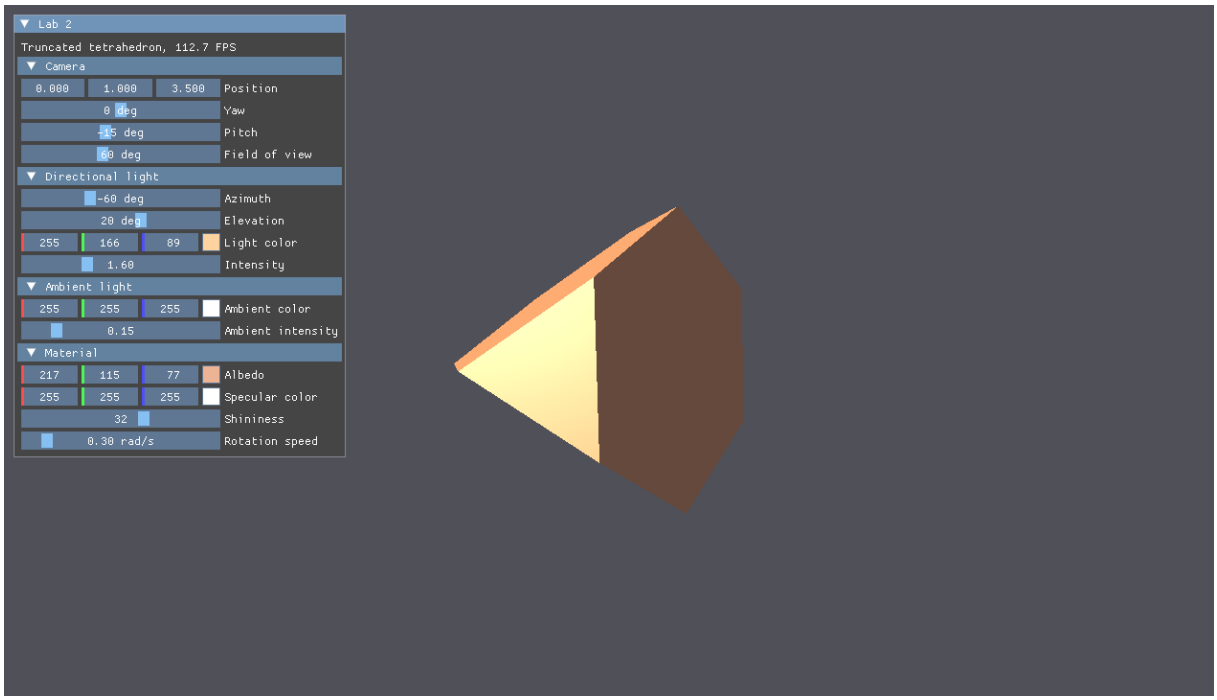


Figure 2 — changed light direction and color

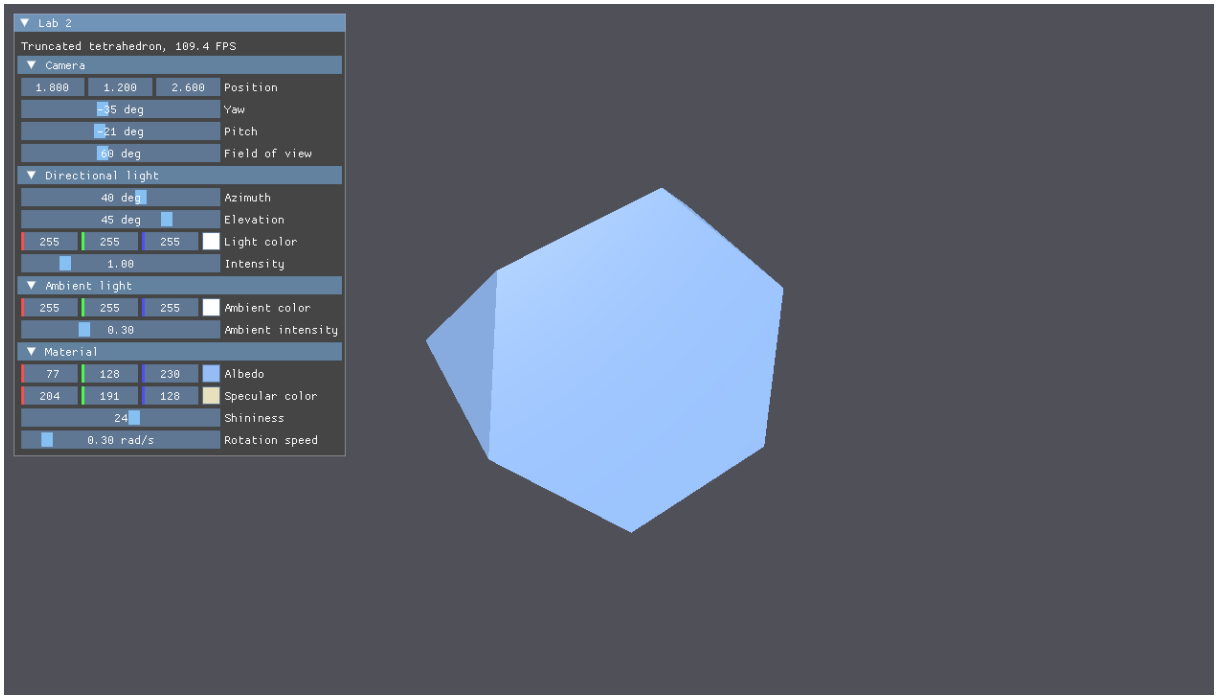


Figure 3 — changed camera position and material specular parameters

Conclusions

I learned to build the view matrix from the camera position and rotation, generate face normals, move normals to world space with the normal matrix and implement the Blinn-Phong lighting model with ambient light and a directional light in the fragment shader.